Finishing up with Functions

CSL100

September 4, 202024

This Class

- Nested function calls
- Predefined functions
- Macros
- Argument passing

Nested Function Calls

- Functions can call each other
- A declaration or definition (or both) must be visible before the call
  - Help compiler detect any inconsistencies in function use
  - Compiler warning, if both (decl & def) are missing

```
#include<stdio.h>
int min(int, int); //declaration
int max(int, int); //of max, min

int max(int a, int b) {
  return (a > b) ? a : b;
}

// a "cryptic" min, uses max
int min(int a, int b) {
  return a + b – max (a, b);
}

int main() {
 printf("%d", min(6, 4));
}
```

Sample Question: Euclid's Number

- The n-th *primorial* number, pn is the product of the first n primes.
  - E.g. p1 = 2, p2 = 6, p3 = 30, and so on.
- The n-th *Euclid* number, En is defined as En = pn + 1.
  - E.g. E1 = 3, E2 = 7, E3 = 31, and so on.
- Question: Write a program that displays the first n Euclid numbers and states for each number whether it is prime or composite.

Solving for Euclid's Numbers

- We can use the top down approach to solve the problem.
- Assume a function to display Euclid numbers (with prime/composite-ness) and use it in main.

```
#include <stdio.h>
void display_euclid(int n); // decl. only
int main(){
    int val;
    scanf("%d",&val);
    display_euclid(val);
    return 0;
}
```

- Now fill in the details of display_euclid
  - May require more assumed functions!

```
int primorial(int n);
int is_prime(int n);

// display first n the Euclid number and whether they
// are prime or composite.
void display_euclid(int n){
    int i, en;

    for (i=1; i<=n; i++) {
        en = primorial(i)+1;
        if (is_prime(en) == 1)
            printf("%d: Prime\n", en);
        else
            printf("%d: Composite\n", en);
    }
}
```

- Keep filling the details of assumed functions
  - Till bottom out (where no more functions are assumed)

```
// return the nth primorial no.
int primorial(int n)
{
    int count=0, i=2, prod=1;
    while (count<n){
        if (is_prime(i) == 1){
            prod = prod * i;
            count++;
        }
        i++;
    }
    return prod;
}
```

```
// return 1 if n is prime,
// else return 0.
int is_prime(int n){
    int i;
    for (i=2; i*i<=n; i++){
        if (n%i == 0)
            return 0;
    }
    return 1;
}
```

```c
#include <stdio.h>

void display_euclid(int n);
int main(){
    int val;
    scanf("%d",&val);
    display_euclid(val);
    return 0;
}

int primorial(int n);
int is_prime(int n);
// display first n the euclid number …
void display_euclid(int n){
   int i, en;
   for (i=1; i<=n; i++) {
       en = primorial(i)+1;
       if (is_prime(en) == 1)
           printf("%d: Prime\n",en);
       else
           printf("%d: Composite\n",en);
   }
}
```

```c
// return the nth primorial no.
int primorial(int n)
{
    int count=0, i=2, prod=1;
    while (count<n){
        if (is_prime(i) == 1){
            prod = prod * i;
            count++;
        }
        i++;
    }
    return prod;
}

// return 1 if n is prime …
int is_prime(int n){
    int i;
    for (i=2; i*i<=n; i++) {
        if (n%i == 0)
            return 0;
    }
    return 1;
}
```

General Principle of Program Development

1. **Break up your task into smaller sub-tasks, and those sub-tasks into still smaller sub-tasks and so on until each sub-task is easily solvable in a function/block.**

2. **Write a function for each of the sub-tasks.**

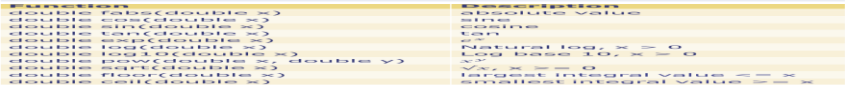
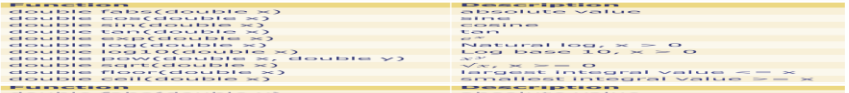
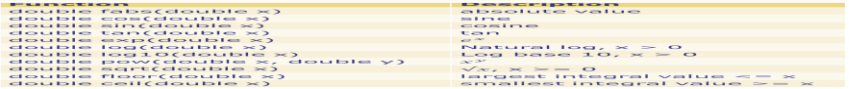
3. **Design your program from the top-down, big task to the small tasks.**

   I. **Debug/test your program bottom-up.**
   II. **Debug functions that perform elementary tasks, and then move on to testing more complex functions.**

Predefined Functions

- C has many predefined functions. We have seen scanf, printf
- To use a predefined function, the corresponding header file must be included
  - Input/Output functions in stdio.h
  - Mathematical functions defined in the library math.h
  - String functions in string.h

Some Predefined Math Functions

Function	Description
double fabs(double x)	absolute value
double cos(double x)	cosine
double sin(double x)	sine
double tan(double x)	tan
double exp(double x)	
double log(double x)	Natural log, x > 0
double log10(double x)	Log base 10, x > 0
double pow(double x, double y)	
double sqrt(double x)	
double floor(double x)	largest integral value <= x
double ceil(doublex)	smallest integral value >= x

Scope of a Name

- Two variables can have the same name only if they are declared in separate scopes
- A variable can not be used outside its scope
- C program has
  - function/**block** scope
  - **file** scope
  - **global**/external scope

Local Variable

- Always declared within a function
- Also called *automatic variable*

auto int a;

- The assigned value is lost when the function is exited
- Could also be declared in a single compound statement

Global Variable

- Variable declared outside every function definition
- Can be accessed by all functions in the program that follow the declaration
- Also called *External* variable
- What if a variable is declared inside a function that has the same name as a global variable?
  - The global variable is "shadowed" inside that particular function only

Global Variables

```
#include<stdio.h>
int g=10, h=20;

int add(){
  return g+h;
}

void fun1(){
  int g=200;
  printf("%d\n",g);
}

int main(){
  fun1();
  printf("%d %d %d\n",
          g, h, add());
  return 0;
}
```

```
200
10 20 30
```

1. The variable g and h have been defined as global variables.
2. The use of global variables is normally discouraged. Use local variables of functions as much as possible.
3. Global variables are useful for defining constants that are used by different functions in the program.

Global Variables: Example

```
const double PI = 3.14159;
double circum_of_circle(double r) {
   return 2 * PI * r; }
double area_of_circle (double r) {
   return PI * r * r;
}
```

defines PI to be of type double with value 3.14159. Qualified by const, which means that PI is a constant. The value inside the box associated with PI cannot be changed anywhere.

Constants via #define

```
#define PI  3.14159
double circum_of_circle(double r) {
   return 2 * PI * r; }
double area_of_circle (double r) {
   return PI * r * r;
}
```

replaces PI by the constant 3.14159 everywhere

#define *name replacement-text*

Static Variables

- We have seen two kinds of variables: local variables and global variables.
- There are static variables too.

```
int f () {
  static int ncalls = 0;
  ncalls  = ncalls + 1;
/* track the number of
times f() is called  */
  … body of f() …
}
```

- Use a local variable?
  - gets destroyed every time f returns
- Use a global variable?
  - other functions can change it! (dangerous)

GOAL: count number of calls to f()

SOLUTION: define ncalls as a static variable inside f().

It is created as an integer box the first time f() is called.

Once created, it never gets destroyed, and retains its value across invocations of f().

It is like a global variable, but visible only within f().

Static variables are not allocated on stack. So they are not destroyed when f() returns.

Summary

- Local Variable
  - Visible in scope
  - Lives in scope (destroyed at the point where we leave the scope)
- Global Variable
  - Visible everywhere
  - Lives everywhere (never destroyed)
- Static Variable
  - Visible in Scope
  - Lives everywhere! (but can not be accessed outside scope)

Macros

- Everything you write outside functions is a command to the C pre-processor
- All directives to pre-processor begin with #
- All other pre-processor statements involve using #define
  - To define macros and constants
- A C macro is a rule that maps an input sequence to an output sequence
  - Before the program has compiled

Macros

- Object-like macros
  - #define BUFFER_SIZE 1024
  - Pre-defined macros in C, e.g. _DATE_ , _TIME_ etc.
    - printf("%s\n", __DATE__);
    - printf("%s\n", __TIME__);
- Function-like macros
  - #define min(X, Y) ((X) < (Y) ? (X) : (Y))
    - x = min(a,b) will be expanded to

      x = ((a) < (b) ? (a) : (b))

      Before compilation

Macro Pitfalls

- Macro is a simple copy-paste
- Without parentheses, can make operator precedence betray your code's logic

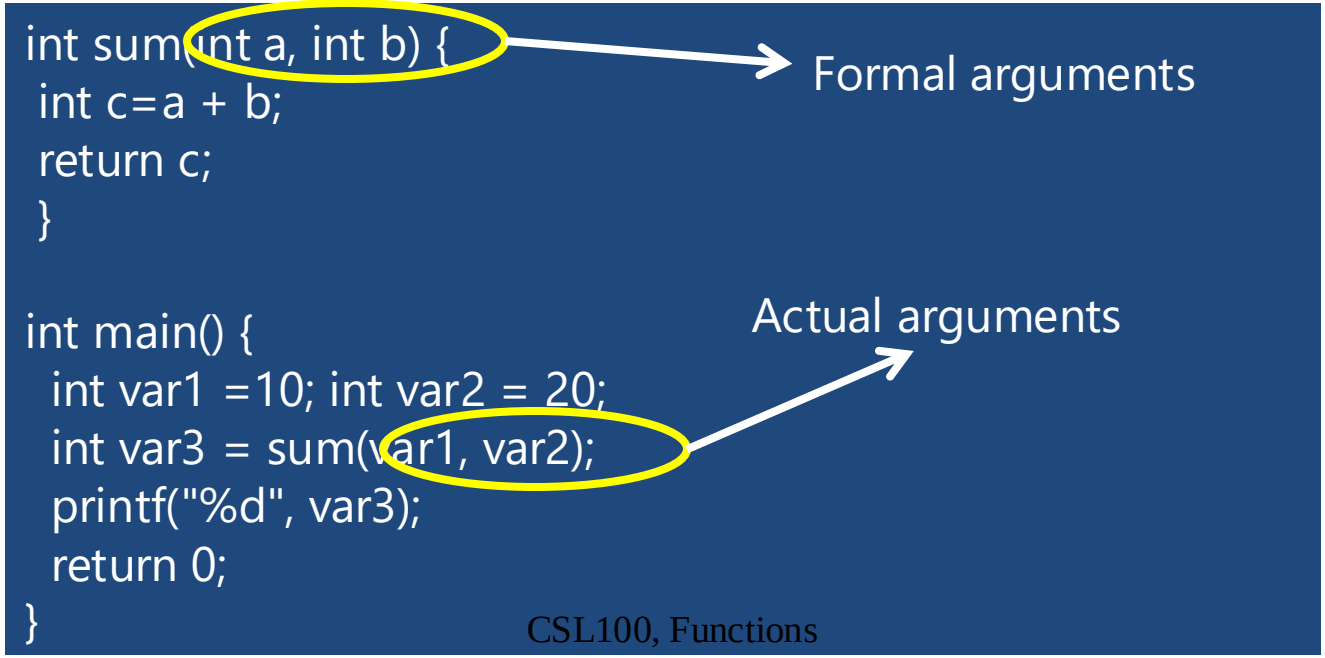
```
#define SQR(x) (x*x)

int main() {
  int a, b=3;
  a = SQR(b+5); // (b+5*b+5)
  printf("%d\n", a);
  return 0;
}
```

23

Argument Passing: Some Details

- Formal argument
  - What you put in the function definition
- Actual argument
  - What you put in the function call



Passing Parameters by Value

- Copy of variable is created in the function's local scope
  - All changes are made to this local variable
  - Global variable retains its original value

```
int increment(int var) {
    var = var+1;
    return var;
}

int main() {
    int num1=20;
    int num2 = increment(num1);
    printf("num1 value is: %d", num1);
    printf("num2 value is: %d", num2);
    return 0;
}
```

```
20
21
```

Another Example

- What will this program output?

```
int main( ) {
    int num1 = 35, num2 = 45 ;
    printf("Before swapping: num1 = %d \t num2 = %d", num1, num2);
    swapnum ( num1, num2 ); //calling swap function
    printf("After swapping: num1 = %d  and num2 = %d", num1, num2);
}

void swapnum ( int var1, int var2 ) {
    int tempnum ;
    tempnum = var1 ;
    var1 = var2 ;
    var2 = tempnum ;
}
```

```
Before swapping: num1 = 35 and num2 = 45
After swapping: num1 = 35 and num2 = 45
```

Passing Parameters by Address

- *Value* of actual argument not passed to formal argument
- *Address* of actual argument passed to formal argument
- Updates value of global variable

An Example

- Remember: & means ‘address of’
- New concept: * means ‘value from’

```
int increment(int *var) {
   *var = *var+1;
    return *var;
}

int main() {
   int num1=20; int num2 = increment(&num1);
   printf("num1 value is: %d", num1);
   printf("num2 value is: %d", num2);
   return 0;
}
```

```
21
21
```

Value from what?

Passing Arguments by Address

- Use pointers as formal arguments
  - Telling compiler you will be passing a memory address, not a value
  - Pass address using *address of* operator (&) during function call

Swap Program Corrected

```
int main( ) {
    int num1 = 35, num2 = 45 ;
    printf("Before swapping: num1 = %d and num2 = %d", num1, num2);
    swapnum ( &num1, &num2 ); /*calling swap function*/
    printf("After swapping: num1 = %d and num2 = %d", num1, num2);
 }

void swapnum ( int *var1, int *var2 ) {
    int tempnum ;
   tempnum = *var1 ;
   *var1 = *var2 ;
   *var2 = tempnum ;
 }
```

```
Before swapping: num1 =35 and num2 = 45
After swapping: num1 =45 and num2 = 35
```

Summary

- Passing arguments by value
  - Global variables unaffected
- Passing arguments by address
  - Global variables potentially affected

Next Class

- Recursion

